

Jumping Mechanics: Kinematic Analysis

Emma Deskin, Carolina Ferreira Silva, Shea Ivey, Ashton Lambert

University of Arizona

BME 214 - Introduction to Biomechanics

Dr. Arellano

October 03, 2023

Objectives

By using a 2D motion capture system known as Kinovea, a human jump can be analyzed at the joints and therefore quantified for data collection. The data can then be converted and employed in predicting maximum jump heights from various initial conditions. Errors and discrepancies are also identified to analyze the difference in prediction and actual performance.

Methods

In order to measure our subject's vertical displacement upon 3 different jumps: countermovement jump, non-countermovement, and single leg countermovement, our group decided to track the hip joint for several reasons. Firstly, using the hip as the reference provides a point closest to the center of mass of the body compared to the knee or ankle joints. Being close to the center of mass leads to maximized accuracy when it comes to calculating the height of each jump. In order to ensure consistency, the hip was tracked in each trial. Furthermore, tracking the hip can provide insight into the biomechanics of the jump. It allows you to assess the athlete's technique and form during the jump, which can be useful for identifying areas for improvement and optimizing performance.

To enable the tracking of multiple jump trials with precision, a consistent setup was imperative. A camera was positioned in front of a white wall, upon which 10 cm intervals were marked with blue tape. The Kinovea software was calibrated to these marked heights, allowing for subsequent measurement of jump height and velocity in each video recording. The tape markings served as a reference for determining true height and acted as supplementary validation alongside the Kinovea software's results. In order for the software to read an accurate height, the subject performed the jumps standing as close to the wall as possible. Significant distances

between the subject and the wall would have introduced measurement errors. Originally, the camera was set up 0.3 meters above the floor on the default frame rate for the videos.

One group member performed the 3 different jumps, countermovement jump, non-countermovement, and single leg countermovement. Upon data collection using Kinovea, it became evident that experimental errors had occurred, resulting in an unexpectedly recorded jump height of approximately 57 cm which contradicted visual observations of the subject's vertical jump. To address this discrepancy, adjustments were made to the experimental setup.

Firstly, it was determined that the low camera angle had distorted the data by elongating the calibrated heights. In order to fix this, the camera was placed at a height closer to the hip, where the body's center of mass is. Along with this, a leveler was used to verify that the footage came out to be as straight as possible. Given that the hip was the subject of tracking, the videos produced at this height provided a much higher level of accuracy.

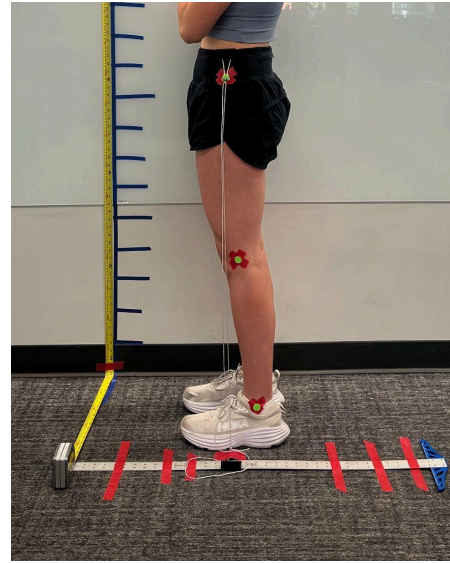


For further clarification on the height traveled, a new form of measurement, a



string-pulley system, was added to the experiment. The end of a string was fixed to the ground, while a binder clip held the tense string affixed to the subject's hip. When completing a jump, the string extended through the clip with the same distance traveled by the subject. We were able to match this physical measurement with the digital

data acquired by the Kinovea program by precisely measuring the distance the pulley system moved as the subject jumped. Furthermore, the spring pulley system provided a precise and consistent method of determining jump height across a number of attempts. It removed any potential differences in the subject's jumping style or the impact of the landing, which could have a negative impact on the precision of height readings. This consistency was crucial for generating accurate and comparable results, particularly when analyzing variations in jump ability over time or between different people.



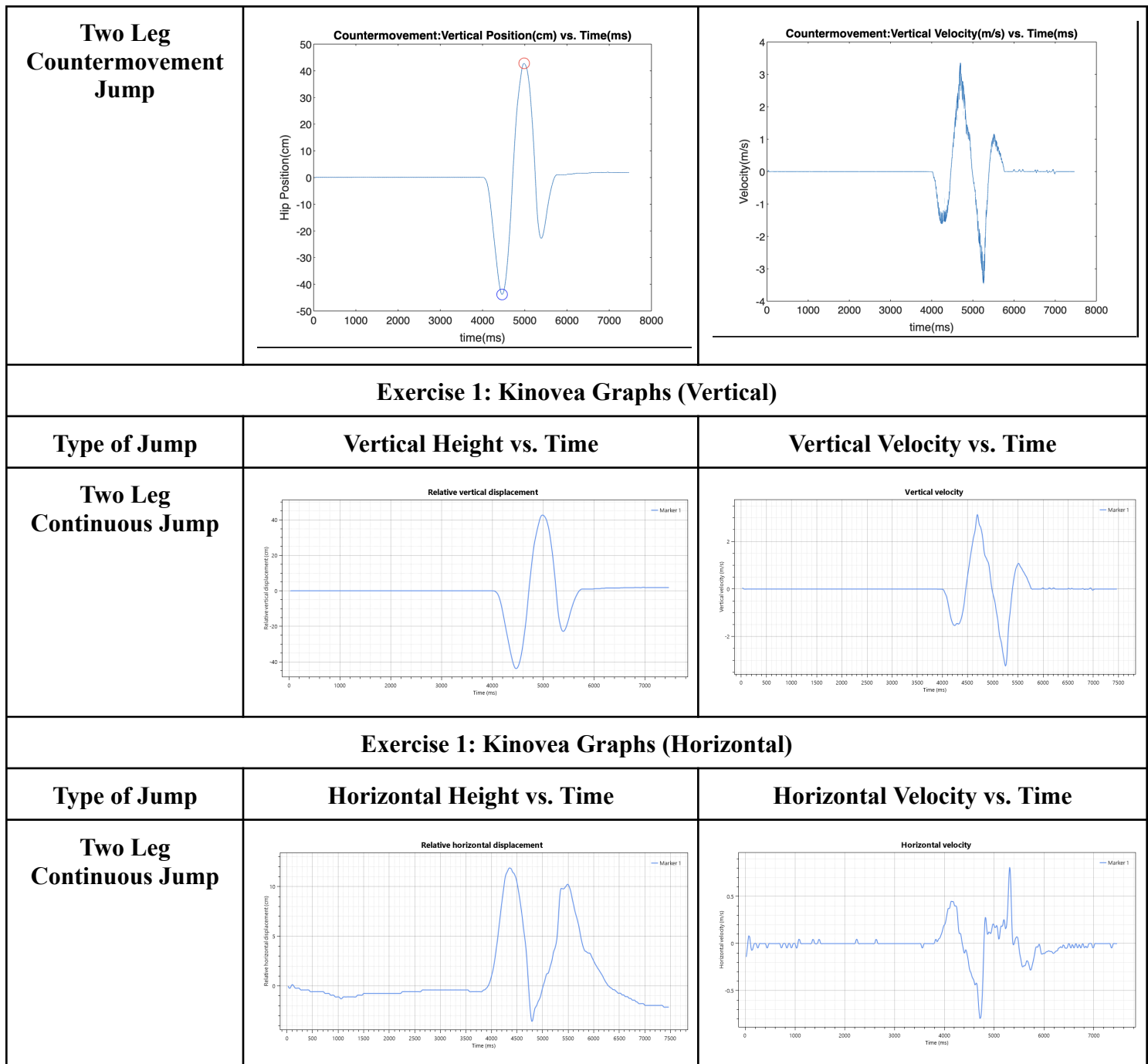
Exercise #1

Track a point on the subject's hip during a countermovement jump to determine vertical and horizontal movement. See data in Table 1 and graphs from Matlab and Kinovea for reference below.

Exercise 1: Data		
Type of Moment:	Countermovement Jump	
String Height:	41 cm	
Subject:	Emma Deskin	
Subject Height (cm)	172.5	
Standing Hip Height (cm)	Minimum Hip Height (cm)	Maximum Hip Height (cm)

121.02	77.2569 (-43.7631 cm bend)	163.7561 (42.7361cm jump)
Vertical Velocity at Standing Height (m/s) (KINOVEA)	Vertical Velocity at Takeoff (m/s) (MATLAB)	Vertical Velocity at Maximum Height (m/s)
2.99878	2.7655	0
Horizontal Velocity at Standing Height (m/s)	Horizontal Velocity at Takeoff (m/s)	Horizontal Velocity at Maximum Height (m/s)
-0.78581	-0.5968	0.2105
Takeoff Time (ms) / Landing Time (ms)	4735	5228
Air Time (ms)	493	
Hip Height (at Takeoff) (cm)	6.1658	
Max Height based on Matlab graph (cm)	42.7361	
Vertical Height based on Initial Vertical Velocity (cm)/ Percent Error	45.1869	5.73%
Vertical Height based on Airtime and Initial Vertical Velocity (cm)/ Percent Error	44.5799	4.31%
Initial Velocity based on Airtime (m/s)	2.4206	
Vertical Height based on Airtime (cm)/ Percent Error	36.0602	15.62%

Exercise 1: Matlab Graphs		
Type of Jump	Vertical Height vs. Time	Vertical Velocity vs. Time



Exercise 1 Discussion

For exercise 1 we have produced the above graphs. The Matlab "Vertical Velocity vs. Time" graph shows how the hip position is changing over time. We used relative position so that the hip starting position is represented at 0. As the subject bent their legs the graph shows a

decrease in hip height. This decrease happens until the subject reaches the minimum bending position which is highlighted by a blue circle on the graph. The minimum hip position is -43.7631 at 4464 milliseconds. After this, the position increases as the subject initiates the jump. During this time, it is important to note that the airtime does not start until the feet leave the ground. We are unable to see the exact moment the feet leave the ground on the position graph, so we used the Kinovea video to manually search frame by frame to determine this. We found this time to be 4735 milliseconds. With this time we were able to use matlab to determine the initial velocity. To do this we found the vertical velocity vs. time graph, by taking the differential of position vs time data. Then, we found the velocity at the exact take off time in order to reduce error. The graph also highlights the maximum hip position with a red dot. The matlab code uses this peak and found that the height the subject jumped during the countermovement was 42.7361 cm at a time of 4982 milliseconds. This position is also when the final velocity equals zero. Finally, the graph shows a steady decrease in position until the subject lands back on the ground. Airtime was calculated from the time the feet left the ground to the time the feet touched back on the ground. These values were found from the video.

With all of the data we collected from kinovea and matlab we were able to make some predicted jump height. These predictions allowed us to verify that the maximum height is correct. The jump height was calculated from the initial velocity, from both the airtime and initial velocity, and finally using only airtime. The calculations for all of these predictions is shown below in the *Matlab Program & Calculations* section. These predictions along with a percent error calculation are shown in the table above. Overall, we found that for the countermovement jump the Vertical height was most accurately predicted using the airtime and initial velocity.

Having both of these values increased the accuracy of the prediction. This came out to a mere 4.31% error.

Another interesting take away from the graphs is that the maximum velocity occurs prior to the takeoff position. When examining the data, it is seen that the velocity of the subject when passing the initial hip height is greater than the velocity at takeoff. This could be explained by the fact that there is more muscle tension and potential energy being used by these muscles as the subject is still in the process of extending their legs to take off. After this energy has been consumed and muscle tension released, the velocity decreases, then upon takeoff continues to do so at a greater rate due to the force of gravity. An Olympic athlete could use this information in that they need to maximize the force exerted by their lower extremity muscles in order to supply a greater velocity as they extend upwards, focusing more on the motion upwards than the takeoff itself.

Exercise #2

Track a point on the subject's hip during a *non*-countermovement jump to determine vertical and horizontal movement. See data in Table 2 and graphs from Matlab and Kinovea for reference below.

Exercise 2: Data	
Type of Moment:	<i>Non-Countermovement Jump</i>
String Height:	<i>33 cm</i>
Subject:	<i>Emma Deskin</i>

Standing Hip Height (cm)	Minimum Hip Height (cm)	Maximum Hip Height (cm)
121.02	79.707 cm (41.313 cm bend)	156.238 cm (35.218 cm jump)
Vertical Velocity at Standing Height (m/s) (KINOVEA)	Vertical Velocity at Takeoff (m/s) (MATLAB)	Vertical Velocity at Maximum Height (m/s)
2.9226	2.4868	0
Horizontal Velocity at Standing Height (m/s)	Horizontal Velocity at Takeoff (m/s)	Horizontal Velocity at Maximum Height (m/s)
-0.78788	-0.52264	-0.05196
Takeoff Time (ms) / Landing Time (ms)	6442	6888
Air Time (ms)	446	
Hip Height (at Takeoff) (cm)	6.2987	
Max Height based on Matlab graph (cm)	35.218	
Vertical Height based on Initial Vertical Velocity (cm) / Percent Error	37.8506	7.48%
Vertical Height based on Airtime (cm) and Initial Vertical Velocity (m/s) / Percent Error	37.729	7.13%
Initial Velocity based on Airtime(m/s)	2.3324	
Vertical Height based on Airtime (cm)/ Percent Error	34.0542	3.30 %

Exercise 2: Matlab Graphs

Type of Jump	Vertical Height vs. Time	Vertical Velocity vs. Time
Two Leg Non- Countermovement Jump		
Exercise 2: Kinovea Graphs (Vertical)		
Type of Jump	Vertical Height vs. Time	Vertical Velocity vs. Time
Two Leg Non- countermovement Jump		
Exercise 2: Kinovea Graphs (Horizontal)		
Type of Jump	Horizontal Height vs. Time	Horizontal Velocity vs. Time
Two Leg Non- Countermovement Jump		

Exercise 2 Discussion

For exercise 2 we were able to use Matlab to create the same graphs as in exercise 1. This allows us to compare the data. For the non-countermovement, the position graph starts out at a

hip position of zero just like the countermovement. Then, it declines as the subject bends their knees. Once at the bottom the graph plateaus. This shows the time when the subject held their position before accelerating upward. Then, once the subject jumps we can first notice that the maximum jump height for the non-countermovement is only 35.218 cm. This is a decrease of 7.5181 cm compared to the countermovement jump. Using this data we can conclude that fluid jumping movements produce greater jumping force and better performance than non-countermovement jumps. The decrease in performance can also be attributed to the decrease in take-off velocity. The countermovement jump had an initial take-off velocity of 2.7655 m/s compared to the non-countermovement jump which was 2.4868 m/s.

To verify our data we completed the same predicted jump height calculations as we did for exercise 1. We predicted this again with initial velocity, airtime and initial velocity, and then finally airtime alone. From these calculations we discovered that the prediction based on airtime alone was the most accurate. Overall, the errors all came out to under 8%. This shows that there is minimal error in our calculated jump height. Verifying our conclusion that the countermovement jump produces a larger jump height compared to the non-countermovement jump.

Horizontal Velocity Analysis for Experiments

As shown in the results of the tests, the subject displayed slight horizontal velocity when completing the jumps. A person's standing vertical jump height can be affected by the presence of small horizontal velocity because it introduces a forward/backward momentum component that competes with the upward force produced during the jump. A person's objective when jumping vertically from a standing position is to maximize the upward power generated by their leg muscles in order to jump as high as possible. Even minimal horizontal movement contributes

a new component to their overall motion. It can become more difficult to reach the same height as a solely vertical jump because the horizontal motion displayed can take part of the energy away from the vertical drive. In all, the subject must combat not only gravity but also the horizontal momentum when reaching for a maximum height.

Matlab Program & Calculations

Sample Matlab Code

In order to analyze our jumping performance our group used Matlab to create graphs and do calculations for each jump (Continuous, Non-continuous, Right-leg, Left-leg). From Kinovea we exported the Vertical Position vs. Time data. Using this data, we created graphs for Vertical Position vs. Time as well as Vertical Velocity vs. Time. The code also calculates all of the data for us, so that we can reduce calculation errors. Below is the code for Exercise 1. The code is the same for the other jumps with the following changes: the data imported, the time the feet left the ground, and the landing time. To accurately determine the takeoff time we went frame by frame on each video to see the exact moment when the feet left the ground and touched the ground. These time values were then coded for each different Exercise.

Matlab Code: Exercise 1

```
%%This is the final- Excercise 1 jump script
data= readmatrix('BME_214/Final_code/cont_data_final.csv');

%creates vertical position vs.time graph
time=data(:,1);
hip=data(:,2);
plot(time,hip)
xlabel("time(ms)");
ylabel("Hip Position(cm)");
title("Countermovement:Vertical Position(cm) vs. Time(ms)");
hold on

%finds the max height index/plots this as a point
[maxValue, maxIndex] = max(hip);
hold on;
plot(time(maxIndex), maxValue, 'ro', 'MarkerSize', 10);
```

```

%finds the minimum height index/plots this as a point
[minValue, minIndex]= min(hip);
hold on;
plot(time(minIndex), minValue, 'bo', 'MarkerSize', 10);
hold off

%finds max height hip value
maxheight=max(hip);
MAXmsg=['The max height Emma got too was ', num2str(maxheight), ' centimeters'];
disp(MAXmsg);

%finds min height hip value
minheight=min(hip);
MINmsg=['The minimum height Emma got too was ', num2str(minheight), ' centimeters'];
disp(MINmsg);

%finds min/max time using index
mintime=time(minIndex);
maxtime=time(maxIndex);

%creates velocity vs time graph (m/s and ms)
velocity = diff(hip/100) ./ diff(time/1000);
figure;
plot(time(2:end), velocity);
vdata=[time(2:end), velocity];
t=vdata(:,1);
vel=vdata(:,2);
xlabel("time(ms)");
ylabel("Velocity(m/s)");

% this is not meters/second
title("Counter movement: Vertical Velocity(m/s) vs. Time(ms)");

%known values
Vf=0;
a=-9.8;

%%take off/landing times based on kinovea timeframe
feetoffground=4735;
feetonground=5228;
takeofftimeMSG=['The take off time is ', num2str(feetoffground) ' ms'];
disp(takeofftimeMSG);
takeofftimeMSG=['The landing time is ', num2str(feetonground), ' ms'];
disp(takeofftimeMSG);
%%calculates time index at takeoff
heightindex=find(time==feetoffground);

%uses time index to find the hip height at takeoff
hipfeetoffground=hip(heightindex);

%calculates the velocity at take off
index_fog=find(t==feetoffground);
velocity_fog=vel(index_fog);
velocityMSG=['The velocity based on when the foot left the ground was ', num2str(velocity_fog), ' m/s'];
disp(velocityMSG);

%%equation used is delta(y)= (finalv^2-initialv^2)/2a
best=((Vf^2)-(velocity_fog^2))/(2*a)+(hipfeetoffground/100);
bestcm=(best*100);

calcheightvelocityMSG=['The calculated maximum height based on the initial velocity is ', num2str(bestcm), ' centimeters'];

```

```

disp(calcheightvelocityMSG);

%displays the tippytoe height
tippytoe=hipfeetoffground;
tippytoeMSG=['The initial height when the feet left the ground was ', num2str(tippytoe), ' cm'];
disp(tippytoeMSG);

%%this code calculates airtime- based on when feet leave/enter ground
fullairtime=feetonground-feetoffground;
airtimeMSG=['The airtime for the jump was ', num2str(fullairtime), ' milliseconds'];
disp(airtimeMSG);

%the time it takes to get to the max height
timetomax=(maxtime-feetoffground);

%height based on airtime and initial velocity  $y = v_i t - \frac{1}{2} a (t^2)$ 
calculatedheight=(velocity_fog*(timetomax/1000))+(.5*a*((timetomax/1000)^2))+(hipfeetoffground/100);
airtime_cm=calculatedheight*100;
calcheightairtimeMSG=['The calculated height based on the airtime and initial velocity is ',
num2str(airtime_cm), ' centimeters'];
disp(calcheightairtimeMSG);

%height based on only airtime=
% $v_f = v_i + at$ , initialvelocity= $v_f - at$ 
initialvelocity=0-(a*(timetomax/1000));
airtimevelocitymsg=['The initial velocity based on airtime is ', num2str(initialvelocity), ' m/s'];
disp(airtimevelocitymsg);

%  $y_{final} = v_i(t) + \frac{1}{2} a (t^2)$ 
heightbasedtime=(initialvelocity*(timetomax/1000))+(.5*a*((timetomax/1000)^2))+(hipfeetoffground/100);
height_time_cm=heightbasedtime*100;
heightbasedtimemsg=['The maximum height based on the airtime is ', num2str(height_time_cm), ' cm'];
disp(heightbasedtimemsg);

```

Exercise 1: Output

The code outputs the Matlab graphs shown in each exercise section as well as the following:

```

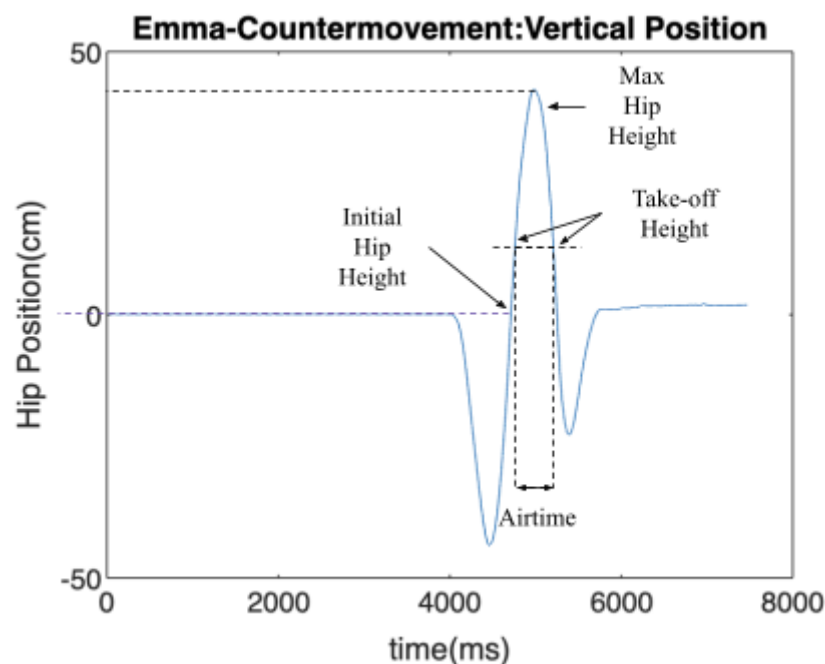
>> Final_continuous_code
The max height Emma got too was 42.7361 centimeters
The minimum height Emma got too was -43.7631 centimeters
The take off time is 4735 ms
The landing time is 5228 ms
The velocity based on when the foot left the ground was 2.7655 m/s
The calculated maximum height based on the initial velocity is 45.1869 centimeters
The initial height when the feet left the ground was 6.1658 cm
The airtime for the jump was 493 milliseconds
The calculated height based on the airtime and initial velocity is 44.5799 centimeters
The initial velocity based on airtime is 2.4206 m/s
The maximum height based on the airtime is 36.0602 cm
>>

```

Having all of the calculations done on Matlab was extremely helpful. For the other three jumps, all we had to do was change the data, take off time, and landing time values. This helped reduce errors, organize, and simplify the experiment.

Calculations

In order to perform the desired calculations, the initial vertical velocity must be chosen from an instant in the recording. Using this and plugging it into Equation 1, the jump height can be solved for. The Matlab script below further breaks down the code programmed to calculate this velocity using the data extrapolated from the Kinovea motion capture software. Additionally, graphs are provided to further support the concept and data analysis. Depicted in the annotated graph below is a visual demonstration of air time, maximum hip height, and more importantly how the initial hip height is not equal to the takeoff hip height. This plays a vital role as takeoff occurs on the subject's tip-toes, whereas the initial height is flat-footed.



Maximum Height based on Initial Velocity:

We can determine the maximum height based on the initial velocity using the following kinematic equation:

$$V_{yf}^2 = V_{yi}^2 + 2a(h_f - h_i) \quad \text{Equation 1}$$

Rearrange it to solve for the final height:

$$\frac{V_{yf}^2 - V_{yi}^2}{2a} + h_i = h_f$$

Then we used Matlab to code for all of the needed values:

Initial known values:

```
%known values
Vf=0;
a=-9.8;
```

Matlab code (Exercise 1) for the initial height:

```
%calculates time index at takeoff
heightindex=find(time==feetoffground);

%uses time index to find the hip height at takeoff
hipfeetoffground=hip(heightindex);
|
```

Matlab code (Exercise 1) for the initial velocity:

```
%calculates the velocity at take off
index_fog=find(t==feetoffground);
velocity_fog=vel(index_fog);
velocityMSG=['The velocity based on when the foot left the ground was ', num2str(velocity_fog), ' m/s'];
disp(velocityMSG);
```

Matlab code for the final height:

```
%equation used is delta(y)= (finalv^2-initialv^2)/2a
best=((Vf^2)-(velocity_fog^2))/(2*a)+(hipfeetoffground/100);
bestcm=(best*100);|
```

Maximum Height based on Airtime and Initial Velocity:

We can determine the maximum height based on airtime using the following kinematic equation:

$$h_f = h_i + v_{if}t - .5(a)(t^2)$$

Equation 2

Then we used Matlab to code for all of the needed values:

Matlab code (Exercise 1) for the initial height:

```
%calculates time index at takeoff
heightindex=find(time==feetoffground);

%uses time index to find the hip height at takeoff
hipfeetoffground=hip(heightindex);
|
```

Initial known values:

```
%known values
Vf=0;
a=-9.8;
```

Initial velocity calculation:

```
%calculates the velocity at take off
index_fog=find(t==feetoffground);
velocity_fog=vel(index_fog);
velocityMSG=['The velocity based on when the foot left the ground was ', num2str(velocity_fog), ' m/s'];
disp(velocityMSG);
```

Time between feet off ground and the max height:

```
%take off/landing times based on kinovea timeframe
feetoffground=4735;
feetonground=5228;
takeofftimeMSG=['The take off time is ', num2str(feetoffground) ' ms'];
disp(takeofftimeMSG);
takeofftimeMSG=['The landing time is ', num2str(feetonground), ' ms'];
disp(takeofftimeMSG);
```

Matlab code for final height with airtime and initial velocity:

```
%height based on airtime  $y = V_i t - \frac{1}{2} a t^2$ 
calculatedheight=(velocity_fog*(timetomax/1000))+(.5*a*((timetomax/1000)^2))+hipfeetoffground/100;
airtime_cm=calculatedheight*100;
calcheightairtimeMSG=['The calculated height based on the airtime is ', num2str(airtime_cm), ' centimeters'];
disp(calcheightairtimeMSG);
```

Maximum Height based on Airtime:

To calculate the airtime based on Airtime we had to use two different kinematic equations:

$$v_f = v_i + at$$

$$y_f = y_i + v_i t + (.5 * a * t^2)$$

Firstly, we calculated the initial velocity using the first kinematic equation:

```
%height based on only airtime=
%vf=vi+at, initialvelocity=vf-at
initialvelocity=0-(a*(timetomax/1000));
airtimevelocitymsg=['The initial velocity based on airtime is ', num2str(initialvelocity), ' m/s'];
disp(airtimevelocitymsg);
```

Then finally, we calculate the jump height using the calculated initial velocity and airtime:

```
% y final = vi(time) +1/2a(t^2)
heightbasedtime=(initialvelocity*(timetomax/1000))+(.5*a*((timetomax/1000)^2))+ (hipfeetoffground/100);
height_time_cm=heightbasedtime*100;
heightbasedtimemsg=['The maximum height based on the airtime is ', num2str(height_time_cm), ' cm'];
disp(heightbasedtimemsg);
```

Calculations Discussion

Each different method for predicting jump height had different accuracy outcomes for each of the different jumps (countermovement, non-counterment, right-leg(below), left-leg(below)) . Overall the errors for the prediction using initial velocity came out to 5.73%, 7.48%, 1.60%, and 1.12%. The percent errors for the calculation with initial velocity and airtime came out to 4.31%, 7.13%, 1.65% and 1.91%. Finally, the percent errors for the calculation with airtime came out to 15.62%, 3.30%, 1.31%, and 14.9%. The prediction with the most success came out to be when we used initial velocity and airtime. This is most likely because we did not have to calculate as much. It took in the most data and used it in the prediction. The next most accurate prediction was when we used the initial velocities. This produced results very similar to when we used initial velocity and airtime. The least accurate prediction was from the use of

airtime. This is most likely due to the fact that airtime does not account for the actual initial velocity at all. As well, the times chosen for when the subjects feet came off and on the ground are difficult to precisely analyze. The initial velocity calculated from airtime generally showed lower velocities than the actual initial velocities. This accounts for the lower jump height.

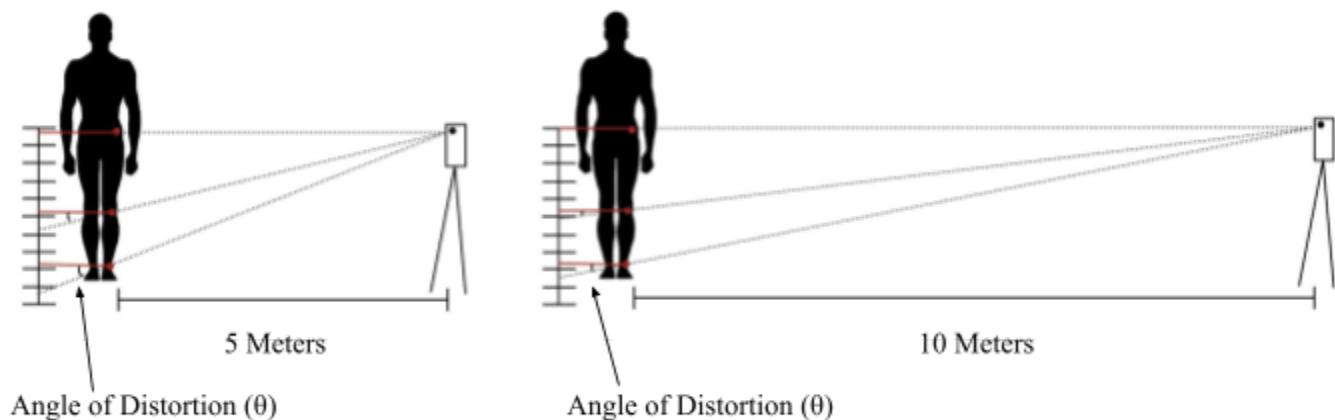
Exercise #3

Replication is key to validating any experimental methods. In the scenario provided, efficient replication is emphasized, in addition to the need to ensure accuracy for high performance athletes. Throughout this project, and further detailed in the methods section above, we employed multiple ways of ensuring the validity of the results captured. The first of such was the use of a string pulley system. This strategy was a quick and quantifiable way to determine the height of a jump, as the string was tense prior to the jump, then after landing the jump, due to the distance extended as the athlete moved past typical standing height, there was slack left in the spring afterwards. Therefore, the difference in distance of the tense string at standing height and the amount of slack string after the jump can be measured. These values coincided with the calculated measurements from the motion capture software, Kinovea.

The string data aligned with our jump height trends. However, there were some discrepancies that can be accounted for. The shorts the subject was wearing were loose. When the subject jumped the string pulled the shorts down a couple of inches. This is consistent with the data which shows lower string length values by a couple of inches. If we were to repeat the experiment we would have the subject wear tighter shorts. As well we would loosen the pulley system slightly, so that the string could easily move with the jumper without resistance.

This leads into the next proof of concept to demonstrate trust in the results provided to the athletes and coaches. The Kinovea software can be calibrated with near precise results. In the images provided of the setup used to record the jump, an enlarged measurement system can be seen. In taking time to mark every 10cm, which is accurate by means of the tape measure additionally depicted next to these markings, the Kinovea can be calibrated with the same such accuracy. Therefore, the measurements collected through Kinovea and the additional string test can be compared. In the above trials for the countermovement jump there was a 6.457% difference in the two measurements, and with the non-countermovement jump there was a 6.502% difference. These findings further implicate that the data is accurate and can be trusted.

It should also be noted that in this scenario with US Olympic coaches, there would be ample room to further ensure accurate data. One such way to eliminate camera distortion depending on the angle of the imaging system is to have the camera far away from the jumper, and then to zoom in on the subject. With the camera being further from the subject, there is a smaller difference in angle on the joint markers and the camera. See the figure below for a visual depiction. Therefore, the distortion has less effect and can further enhance Kinovea's motion capture software accuracy in its calibrations. Additionally, the subjects need to stand very close to the wall, which also plays a role in limiting distortion.

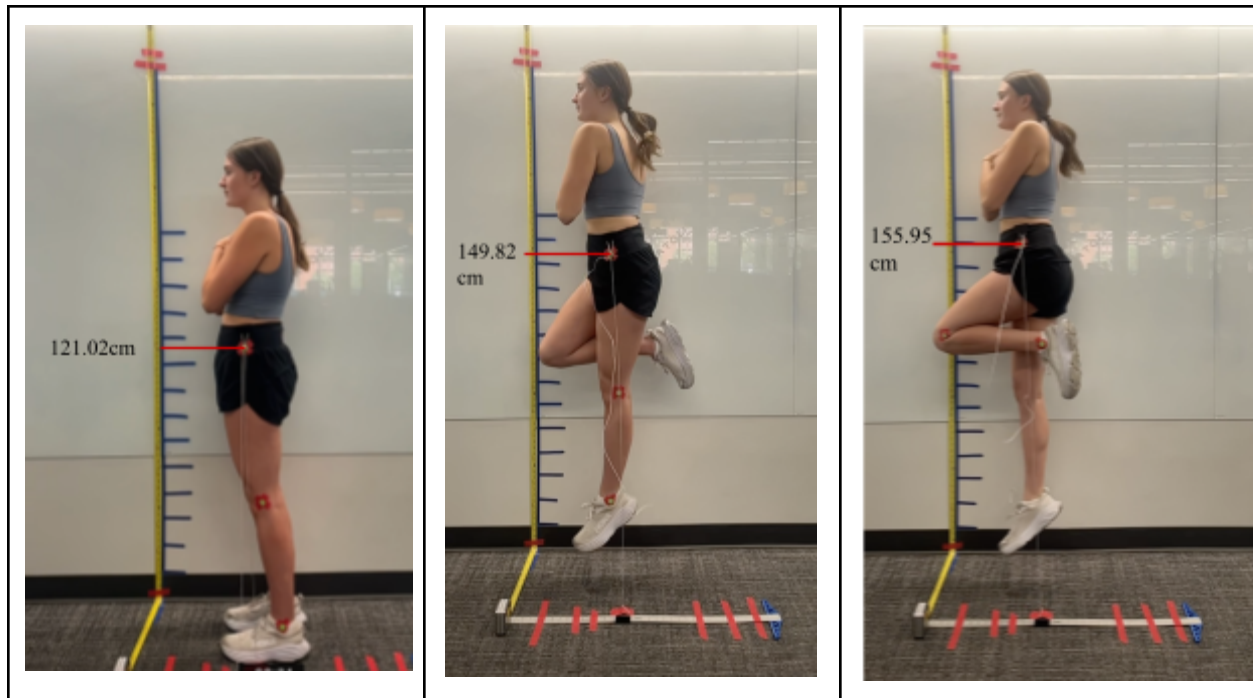


The final aspect of jump performance that was investigated through this project was velocity. A way to prove the validity of this would be to record the jumps with a camera that measures 240 frames per second (FPS). This allows for better results and analysis in the motion capture software because there are more data points to work with and use to calculate instantaneous velocity at various points in time. Furthermore, the velocities measured in Kinovea can then be compared with the velocities calculated using the Matlab program. In the countermovement jump there was a 4.326% percent difference, and for the non-countermovement jump there was a 5.405% percent difference. Therefore, this further demonstrates that the Kinovea coupled with Matlab and higher FPS prove the data provided and analyzed from the jump accurately and completely detail the subject's overall jump performance.

Creative Exercise

Our group decided that we wanted to further analyze the different factors that play a role in jumping. One important factor that stuck out to us was how each leg plays a part in the jumping performance. To analyze this we had our subject do a continuous jump on the right leg and left leg. Below is the setup of these two jumps as well as the data we collected from each:

Creative Jump: Single Leg		
Standing Height	Left Jump at maximum height	Right Jump at maximum height

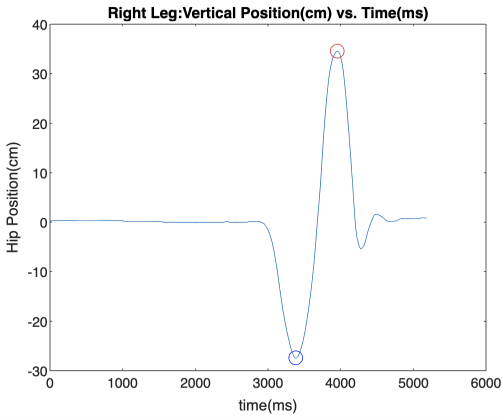
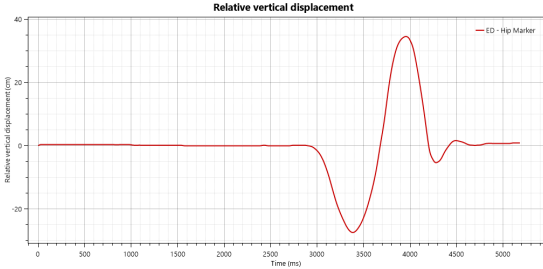


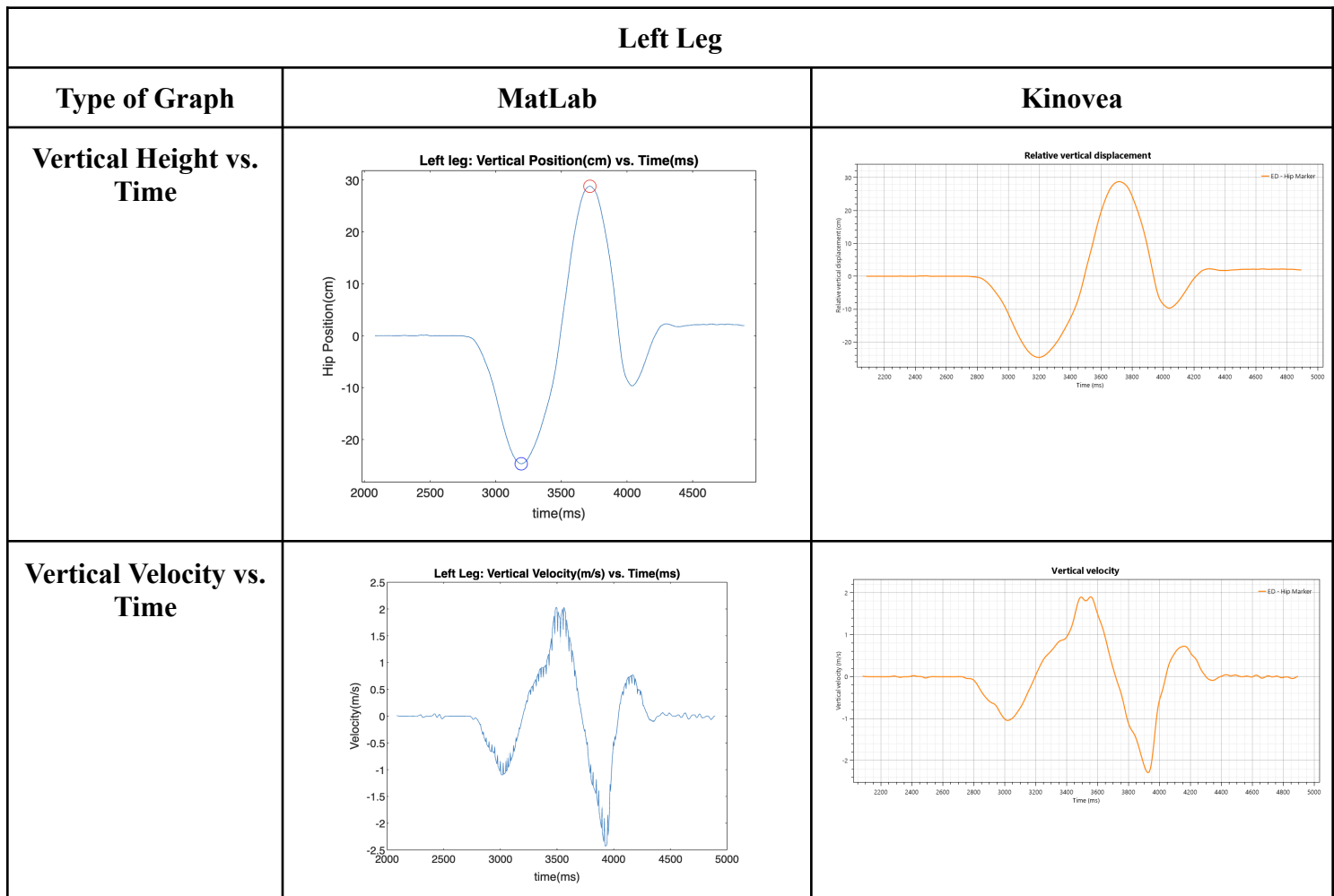
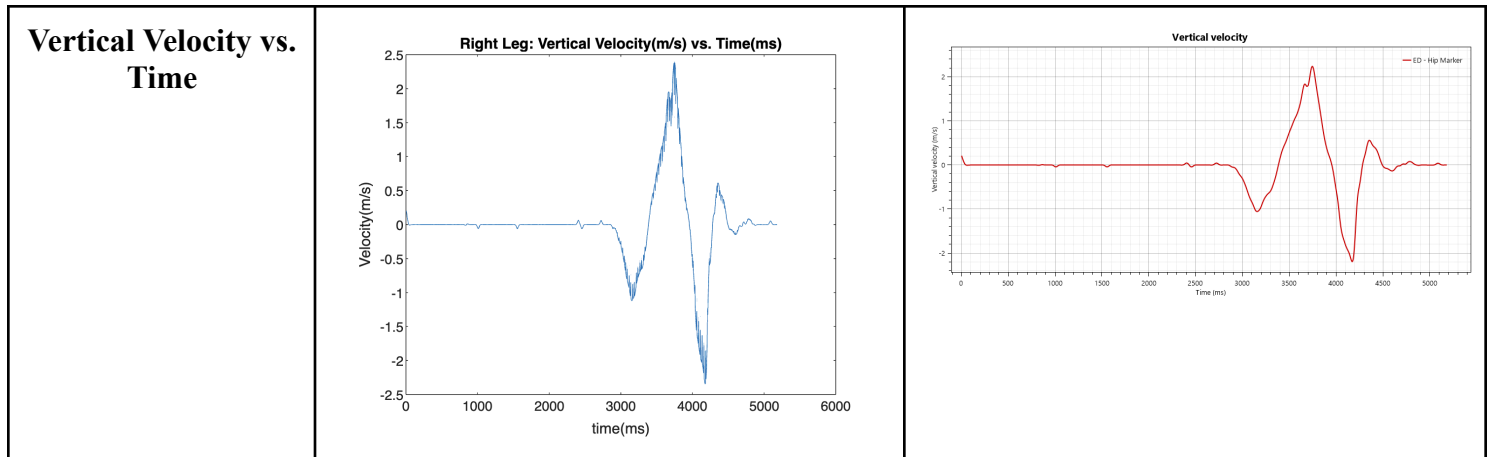
Creative Exercise: Right leg		
Type of Moment:	<i>RIGHT- Countermovement Jump</i>	
String Height:	<i>31.5 cm</i>	
Subject:	<i>Emma Deskin</i>	
Standing Hip Height (cm)	Minimum Height (cm)	Maximum Height (cm)
121.02	93.5662 (-27.4538 cm bend)	155.5597 (34.5397 cm jump)
Vertical Velocity at Standing Height (m/s) (KINOVEA)	Vertical Velocity at Takeoff (m/s) (MATLAB)	Vertical Velocity at Maximum Height (m/s)
1.17747	1.7778	0
Takeoff Time (ms) / Landing Time (ms)	3768	4105

Air Time (ms)	337	
Hip Height (at Takeoff) (cm)	17.8594	
Max Height based on Matlab graph (cm)	34.5397	
Vertical Height based on Vertical Velocity (cm) / Percent Error	33.984	1.60%
Vertical Height based on Airtime and Initial Vertical velocity (cm)/ Percent Error	33.9686	1.65%
Initial Velocity based on Airtime(m/s)	1.8326	
Vertical Height based on Airtime (cm)/ Percent Error	34.9942	1.31%

Creative Exercise: Left leg		
Type of Moment:	<i>LEFT Countermovement Jump</i>	
String Height:	26.5 cm	
Subject:	<i>Emma Deskin</i>	
Standing Hip Height (cm)	Minimum Height (cm)	Maximum Height (cm)
121.02	96.3586 (-24.6614 cm bend)	149.8156 (28.7956 cm jump)
Vertical Velocity at Standing Height (m/s) (KINOVEA)	Vertical Velocity at Takeoff (m/s) (MATLAB)	Vertical Velocity at Maximum Height (m/s)

1.8882	1.937	0
Takeoff Time (ms) / Landing Time (ms)	3542	3880
Air Time (ms)	338	
Hip Height (at Takeoff) (cm)	9.3307 cm	
Max Height based on Matlab graph (cm)	28.7956	
Vertical Height based on Vertical Velocity (cm)/ Percent Error	28.4743	1.12%
Vertical Height based on Airtime and Initial Vertical Velocity(cm)/ Percent Error	28.2445	1.91%
Initial Vertical Velocity based on Airtime (m/s)	1.7248	
Vertical Height based on Airtime (cm)/ Percent Error	24.5089	14.9%

<i>Right Leg</i>		
Type of Graph	MatLab	Kinovea
Vertical Height vs. Time		



Single Leg Creative Jumps Analysis

The subject presents significant asymmetry between dominant and non-dominant legs during single-leg jumps. It is observed that in their dominant leg, the right leg, the subject reached 34.5 cm, while their non-dominant, left leg, jump reached 28.8cm, which is only 81% of the height achieved in their dominant legged jump. Given that current literature considers limb asymmetry to be a difference greater than 10% between limbs, the subject's 19% difference in jump heights is well within those standards.

This limb asymmetry discrepancy occurs when there is a difference in the strength, force, and power between legs, and this value becomes increasingly relevant when discussing performance and injury prevention. There is a strong correlation found in biomechanical studies that indicate that limb symmetry can positively affect the vertical height jump. Through the awareness of this discrepancy, the subject could undergo training to improve their technique and strength training to reduce this discrepancy and achieve optimal results.

There is an active open debate in the orthopedic community attempting to quantify baseline tests for preventing lower limb injury. A hot topic that has come to the media's attention in the past few months is the epidemic of ACL injuries in female athletes; over 25 female footballers were missing from the World Cup due to ACL injuries. Morishige et al. point out that studies have linked leg dominance, which is connected to limb asymmetry, to an elevated risk of ACL injuries in a scientific article published by the Journal of Orthopedic Surgery and Research. This information comes with a strong interest in our subject, a young female recreational athlete. It is in her best interest to improve her limb asymmetry to reduce her risk of lower limb injuries, such as ACL ruptures, and improve her overall jump height performance.

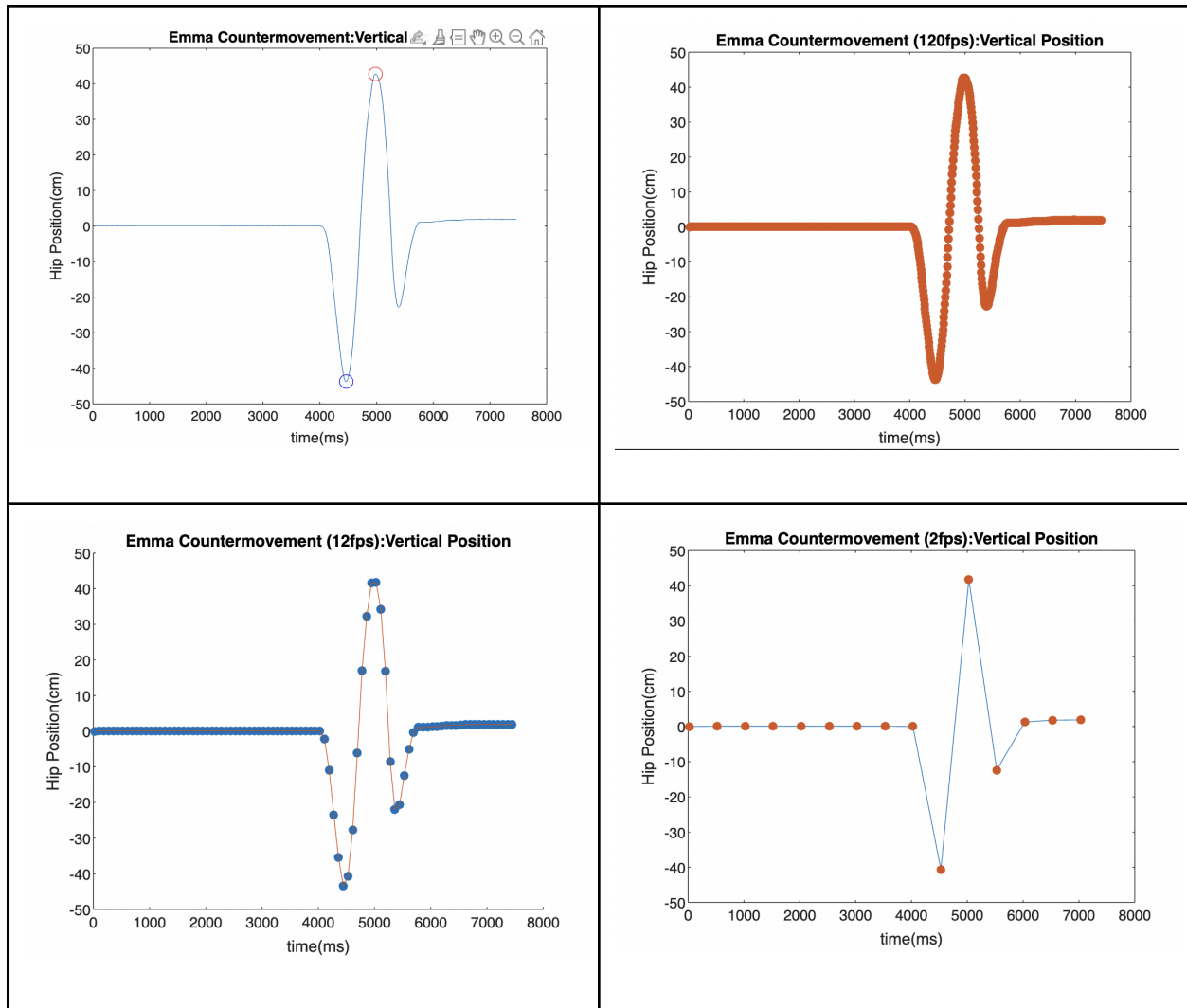
As engineers, we understand the importance of accuracy in data processing, and we all know that the more data you have, the more accurate a result you will achieve. However, working with large amounts of data can be especially problematic regarding data storage and transfer. For instance, we can record the jump video on our phone at 240 fps. However, we ran into several issues, and it took us several attempts to figure out how to deal with such an absurd amount of real data. The struggle made us question the necessity of high sampling frequency for such a simple task. To solve this problem, our group decided to downsample the Kinovea data utilizing MatLab and see how far we could downsample the frequency and still achieve comparable results.

We decided to use our Countermovement jump, which was the jump that provided us with the best data in the original plot that was recorded at 240fps. The data was extremely smooth and did not have a lot of noise, making it ideal for downsampling. Given the length of that video, we ended up with 1786 data points, which ultimately provided us with a highly smooth graph.

Utilizing the downsample function on Matlab, we decided to test 3 frequencies: 120fps, 12fps, and 2fps. Below is the code we wrote to accomplish such a task. We decided to add a scatterplot on top of our plot, which helps us visualize the spread of the data better.

And here are the graphs we produced by changing the frequency of the data including a copy of our original data for comparison.

Downsampled Data Comparisons side by side
--



The idea of downsampling the data into 120 fps, 12 fps, and 2 fps came after extensive research. Initially, we just wanted to reduce the data by half to see how it presented itself. Then we learned that 12fps is commonly used in the animation industry for hand-drawn animation, so we thought it wouldn't be an inadequate representation of a vertical jump. Ultimately, one of our members suggested we try two fps simply to see what it would look like, and surprisingly enough, we still could achieve comparable results through some data manipulation.

Accuracy

There are many ways to tackle downsampled frequency. Initially, we thought of the obvious, utilizing the points provided in the downsampled position vs. time data and seeing how close we could get to the actual takeoff velocity, which was most accurately achieved at 120fps.

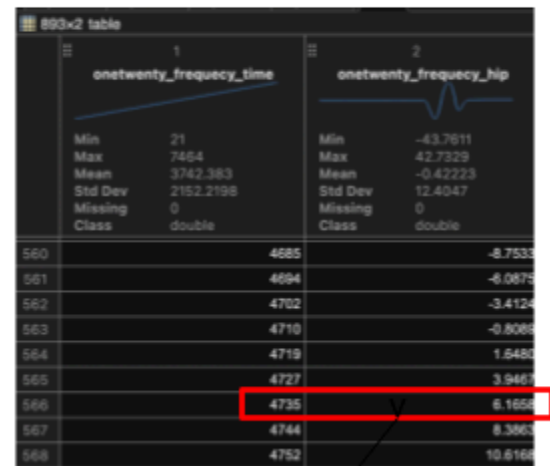
With that downsampled data, we were able to achieve the exact time of take-off which happened at 4735 seconds for the countermovement jump, which is exactly what is utilized to calculate the takeoff velocity. Even after downsampling that data by half, the exact moment the subject begins to undergo acceleration due to gravity is conserved after the downsampling of the data. The take off velocity of 2.7655 can be extrapolated by taking the differential of the position vs time graph at 4735 seconds.

Original Data (240fps)



	1	2
1120	4690	-7.4259
1121	4694	-6.0875
1122	4698	-4.7464
1123	4702	-3.4124
1124	4706	-2.0963
1125	4710	-0.8089
1126	4715	0.4408
1127	4719	1.6480
1128	4723	2.8139
1129	4727	3.9467
1130	4731	5.0596
1131	4735	6.1658
1132	4740	7.2738
1133	4744	8.3863

Downsampled Data (120fps)



	1	2
560	4685	-8.7533
561	4694	-6.0875
562	4702	-3.4124
563	4710	-0.8089
564	4719	1.6480
565	4727	3.9467
566	4735	6.1658
567	4744	8.3863
568	4752	10.6168

Time of takeoff is conserved after downsampling. Leading to the conservation of the take off velocity

Then, although the 2fps graph looks slightly more distorted, it is a pretty accurate representation of our data. Since we did not have any data points in takeoff velocity time, we could have taken the instantaneous rate of change at that time, and that could have provided us with a value that, although it was not presented in the data set, could be extrapolated given the trend of the data and its plot. Remember that this type of extrapolation would only be valid if you already know what to expect and are familiar with the nature of the data. We only recommend utilizing 2fps for proof of concept, or you are downsampling from known data. You already know the takeoff time, or else it would be impossible to predict it just by the data points since, in

our graph, for instance, we get one data point at the lowest position. The next one is at the highest point, making it impossible to predict anything that happened in the frames in between.

What is the optimal sampling frequency?

Ultimately, it depends on your original data. Our initial data did not have any significant outliers and was exceedingly smooth. However, when you downsample noisy data, it comes accompanied by various challenges; for instance, it might inadvertently exaggerate or emphasize noise, resulting in a less accurate representation of the data and loss of meaningful information. Therefore, the optimal sampling frequency is subjective to the original data. However, if the goal is not to downsample the data but to achieve optimal results with a lower frequency rate, 120fps will provide you with significantly accurate results.

MATLAB code utilized for the bonus question.

```
data= readmatrix('Final-ContinuousVertDisplacement.csv');
time=data(:,1);
hip=data(:,2);
%Original Data for comparison
figure(1);
plot(time,hip);
hold on
xlabel("time(ms)");
ylabel("Hip Position(cm)");
title("Emma Countermovement (240fps):Vertical Position")
hold on
hold off
%Reducing frequency to 12fps
twelve_frequecy_time= downsample(time,20);
twelve_frequecy_hip= downsample(hip,20);
T2=table(twelve_frequecy_time,twelve_frequecy_hip);
figure(2);
scatter(twelve_frequecy_time,twelve_frequecy_hip, 'filled');
hold on
plot(twelve_frequecy_time,twelve_frequecy_hip);
hold on
xlabel("time(ms)");
ylabel("Hip Position(cm)");
title("Emma Countermovement (12fps):Vertical Position")
hold off
%Reducing frequency to 2fps
two_frequecy_time= downsample(time,120);
two_frequecy_hip= downsample(hip,120);
T3=table(two_frequecy_time,two_frequecy_hip);
figure(3);
plot(two_frequecy_time,two_frequecy_hip);
hold on
scatter(two_frequecy_time,two_frequecy_hip, 'filled');
hold on
xlabel("time(ms)");
ylabel("Hip Position(cm)");
```

```

title("Emma Countermovement (2fps):Vertical Position")
hold off
%%Reducing frequency to 120fps
onetwenty_frequecy_time= downsample(time,2);
onetwenty_frequecy_hip= downsample(hip,2);
%tabling downsampled data to compare it with og data to find vi
T4=table(onetwenty_frequecy_time,onetwenty_frequecy_hip);
figure(4);
plot(onetwenty_frequecy_time,onetwenty_frequecy_hip);
hold on
scatter(onetwenty_frequecy_time,onetwenty_frequecy_hip,'filled');
hold on
xlabel("time(ms)");
ylabel("Hip Position(cm)");
title("Emma Countermovement (120fps):Vertical Position")
hold off

```

References

Lin, C. Y., Casey, E., Herman, D. C., Katz, N., & Tenforde, A. S. (2018). Sex Differences in Common Sports Injuries. *PM & R : the journal of injury, function, and rehabilitation*, 10(10), 1073–1082. <https://doi.org/10.1016/j.pmrj.2018.03.008>

Morishige, Y., Harato, K., Kobayashi, S. *et al.* Difference in leg asymmetry between female collegiate athletes and recreational athletes during drop vertical jump. *J Orthop Surg Res* 14, 424 (2019). <https://doi.org/10.1186/s13018-019-1490-5>

Rojas-Reyes, Cristian, et al. “Is It Necessary to Normalize Data?” *International Society of Biomechanics in Sports*, Universidad Metropolitana de Ciencias de la Educación, Santiago, Chile, 20 July 2020, commons.nmu.edu/cgi/viewcontent.cgi?article=1926&context=isbs#:~:text=This%20finding%20indicates%20that%20when,through%20this%20type%20of%20applications.